Improving Linux Performance with GCC latest technologies

Victor Rodriguez
SW Engineer
OTC– Advance System Engineering
LPC 2016

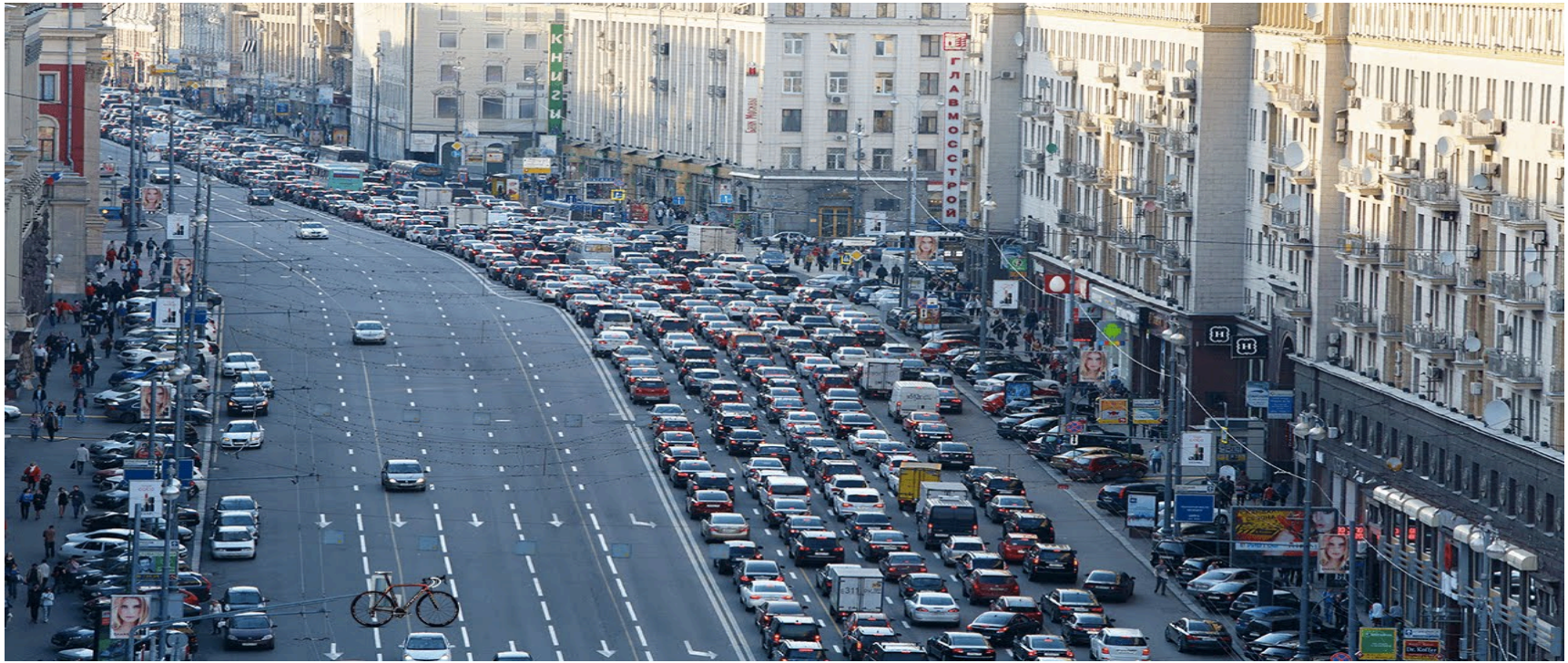


Clear Linux* Project
for Intel® Architecture

Agenda

- Unused resources
- AVX technology
- Function Multi Versioning
- Auto FDO and Profiling

Unused resources….

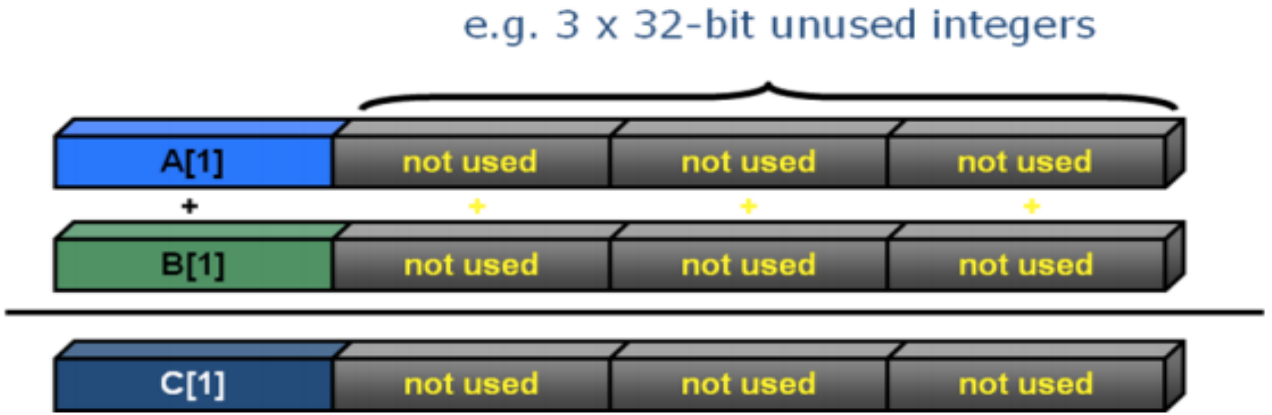


https://i.ytimg.com/vi/X0ymEx64RLw/maxresdefault.jpg

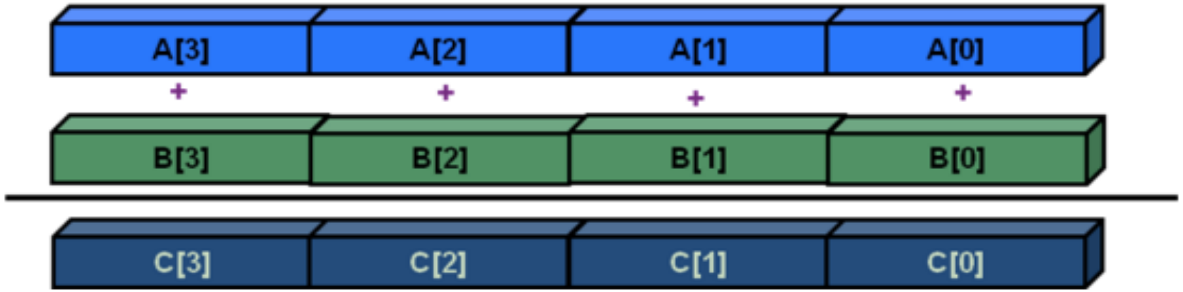
Lets start with something simple

```
#define MAX 1000000
int a[256], b[256], c[256];
int main () {
int i,j;
for (j=0;j<MAX;j++){
       for (i=0; i<256; i++){
                a[i] = b[i] + c[i];
     }
    return 0;
}
```

Vectorization … what is it ?



Vectorization …this is it ?



Vectorization … How to enable?

```
$ gcc -fopt-info-vec sanity.c -O2 -ftree-vectorize
$ gcc -fopt-info-vec sanity.c -O3
```

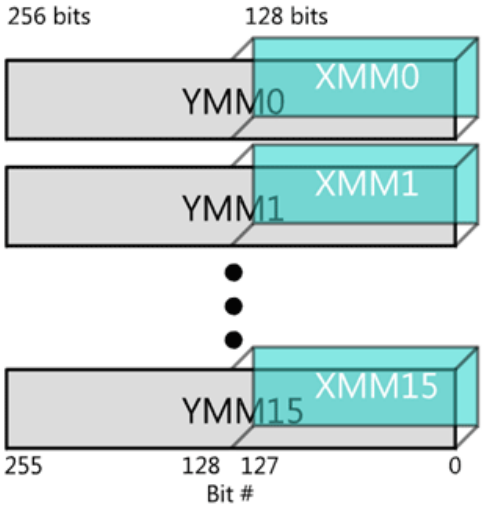
Example	GCC flags	Time
sanity.c	gcc sanity.c -o sanity	603 ms
sanity.c	gcc -O3 sanity.c -o sanity	38 ms

https://github.com/VictorRodriguez/autofdo_tutorial/blob/master/sort.c



HW/SW characteristics

Why don’t we extend the register more ?



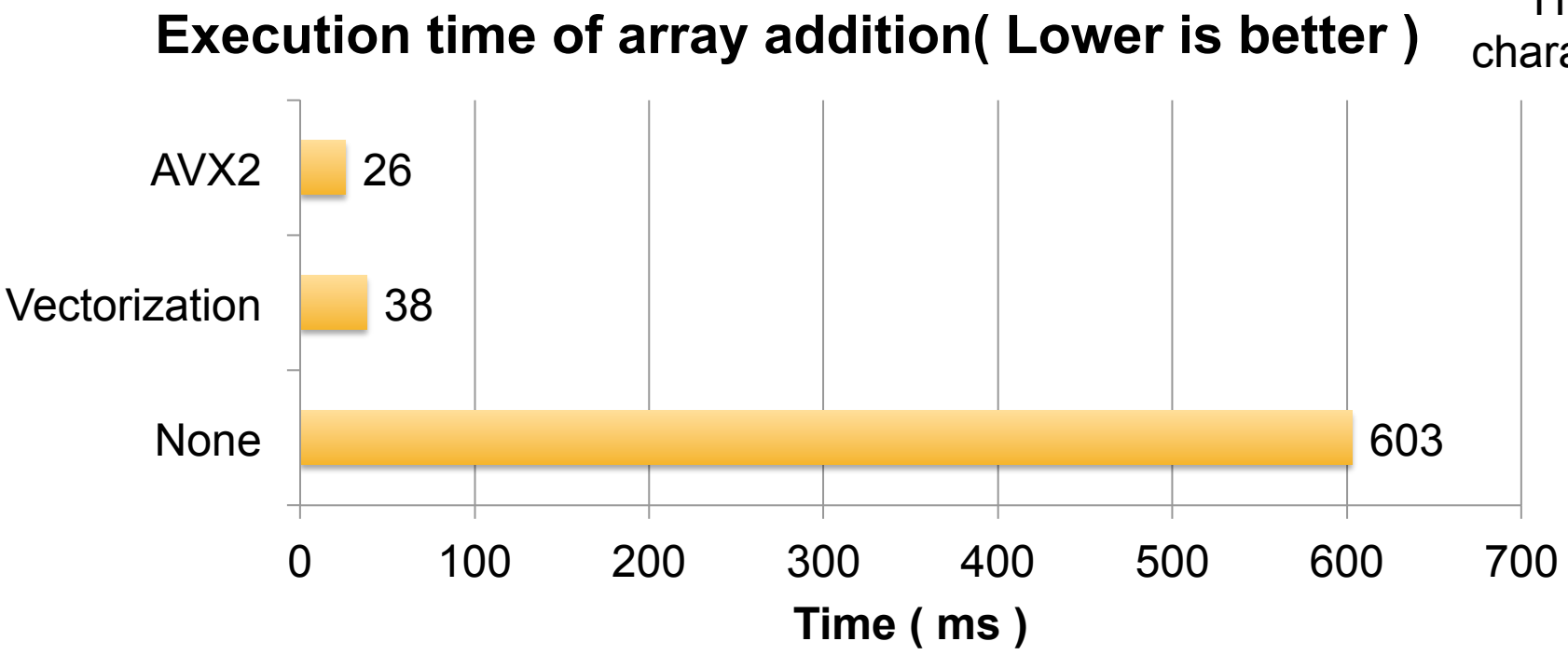
AVX2

```
$ gcc -O3 sanity.c -fopt-info-vec -mavx2 -o sanity
```

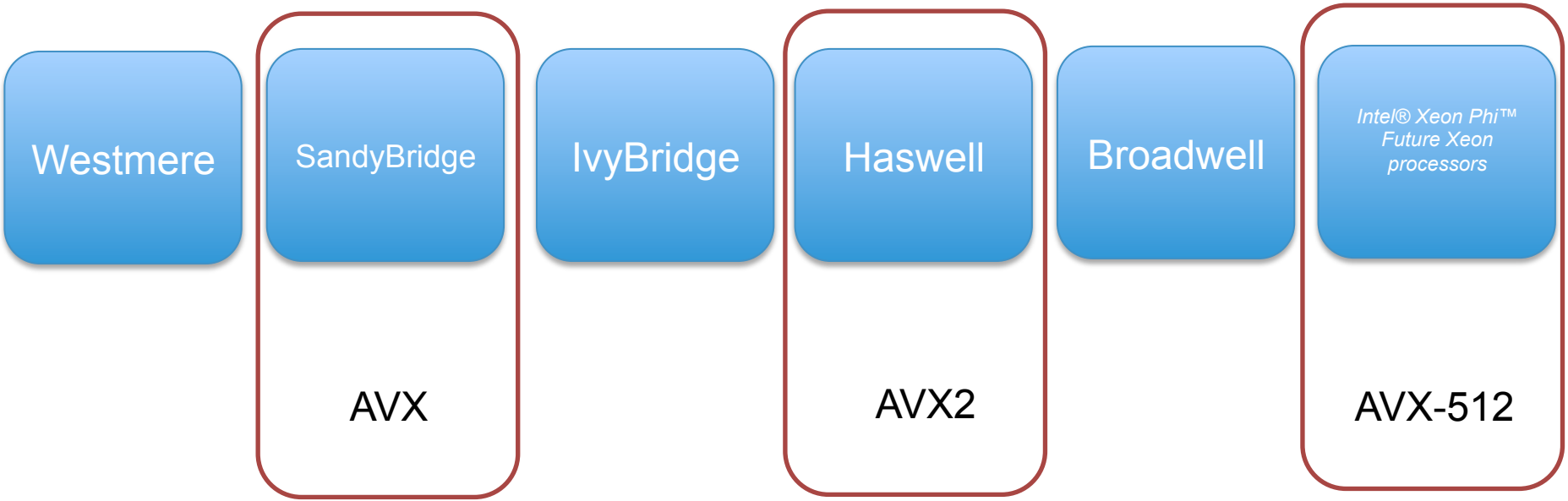
511 -> 256	255 -> 128	127 - > 0
AVX 512	AVX2/AVX	SSE
ZMM0	YMM0	XMM0
ZMM1	YMM1	XMM1
ZMM2	YMM2	XMM2
ZMM3	YMM3	XMM3
ZMM4	YMM4	XMM4
ZMM5	YMM5	XMM5
ZMM6	YMM6	XMM6

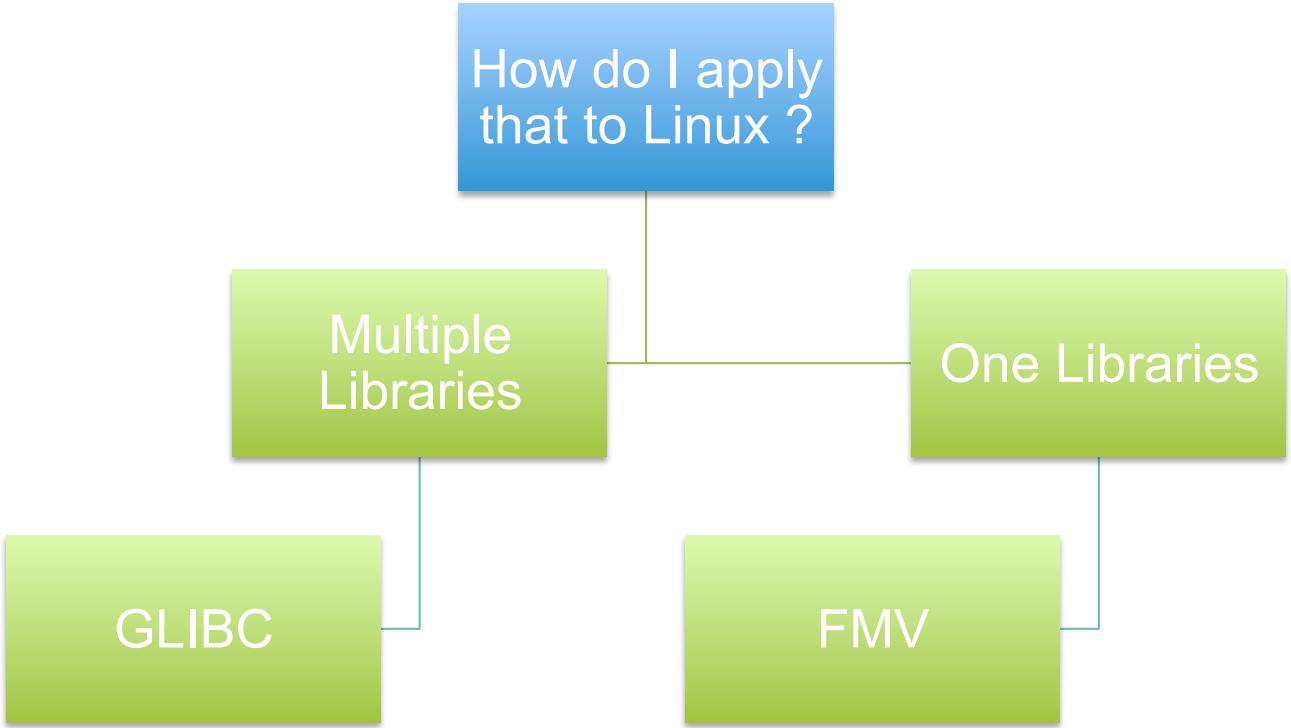
AVX technology advantage

HW/SW characteristics



Intel AVX-512 will be first implemented in the future Intel® Xeon Phi™ processor and coprocessor known by the code name Knights Landing, and will also be supported by some future Xeon processors scheduled to be introduced after Knights Landing




How do I apply that to Linux ?
Multiple Libraries
One Libraries
GLIBC
FMV

GLIBC patch : https://github.com/clearlinux-pkgs/glibc/blob/master/0001-Add-avx2-fake-capability-like-tls.patch

```
+/* Ideally this should simply be __builtin_cpu_supports("avx2") */
+/* However that does not check for OSXSAVE, XSTATE_SSE, XSTATE_YMM */
+/* Origin gcc/testsuite/gcc.target/i386/avx2-check.h */
+/* Origin gcc/testsuite/gcc.target/i386/avx-os-support.h */
+#ifdef __x86_64__
+  unsigned int eax, ebx, ecx, edx;
+
+  if (__get_cpuid (1, &eax, &ebx, &ecx, &edx)) {
+    if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_LIBS))
+      _dl_debug_printf ("  got cpuid\n");
+  } else {
+    if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_LIBS))
+      _dl_debug_printf ("  failed to get cpuid\n");
+    return 0;
+  }
```

+ /* Add 'avx2' capability on x86_64 */

```
+  if (_avx2_available())
+    {
+      if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_LIBS))
+       _dl_debug_printf ("  adding avx2 cap support\n");
+      temp[m].str = "avx2";
+      temp[m].len = 4;
+      ++m;
+    }
+  else {
+    if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_LIBS))
+      _dl_debug_printf ("  not adding avx2 cap support\n");
+    --cnt;
+  }
+
```

Make it work in *ldconfig*

```
--- glibc-2.21.orig/elf/ldconfig.c
+++ glibc-2.21/elf/ldconfig.c
@@ -1297,6 +1297,7 @@ main (int argc, char **argv)
       under which TLS support was optional.  The entry is no longer needed, but
       must remain for compatibility.  */
    hwcap_extra[63 - _DL_FIRST_EXTRA] = "tls";
+  hwcap_extra[62 - _DL_FIRST_EXTRA] = "avx2";
```

In spec files:

https://github.com/clearlinux-pkgs/openblas/blob/master/openblas.spec

```
35  %build
36  export AR=gcc-ar
37  export RANLIB=gcc-ranlib
38  export CFLAGS="$CFLAGS -flto -fno-semantic-interposition -O3 "
39  export FFLAGS="$CFLAGS -flto -fno-semantic-interposition -O3 -fno-f2c "
40  export CXXFLAGS="$CXXFLAGS -flto -fno-semantic-interposition -O3 "
41
42  sed -i -e "s/\-O2/\-O3/g" Makefile*
43
44  pushd ..
45          cp -a OpenBLAS-%{version} openblas-noavx
46          cp -a OpenBLAS-%{version} openblas-avx2
47
48          pushd openblas-noavx
49          make TARGET=SANDYBRIDGE F_COMPILER=GFORTRAN SHARED=1 DYNAMIC_THREADS=1 NUM_THREADS=128 %{?_smp_mflags}
50          popd
51          export CFLAGS="$CFLAGS -march=haswell "
52          export FFLAGS="$FFLAGS -march=haswell -O3 "
53          pushd openblas-avx2
54          make TARGET=HASWELL F_COMPILER=GFORTRAN  SHARED=1 DYNAMIC_THREADS=1 USE_OPENMP=1  NUM_THREADS=128 %{?_smp_mflags}
55          popd
56  popd
57
```

```
%install
rm -rf %{buildroot}
export AR=gcc-ar
export RANLIB=gcc-ranlib
export CFLAGS="$CFLAGS -flto -ffunction-sections -fno-semantic-interposition -O3 "
export CXXFLAGS="$CXXFLAGS -flto -ffunction-sections -fno-semantic-interposition -O3 "

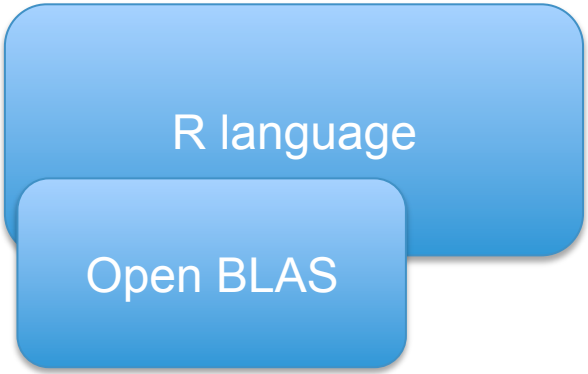
pushd ..

        pushd openblas-noavx
        make install DESTDIR=%{buildroot} PREFIX=/usr OPENBLAS_LIBRARY_DIR=/usr/lib64
        popd
        export CFLAGS="$CFLAGS -march=haswell "
        pushd openblas-avx2
        make install DESTDIR=%{buildroot} PREFIX=/usr OPENBLAS_LIBRARY_DIR=/usr/lib64/avx2
        popd
popd

```

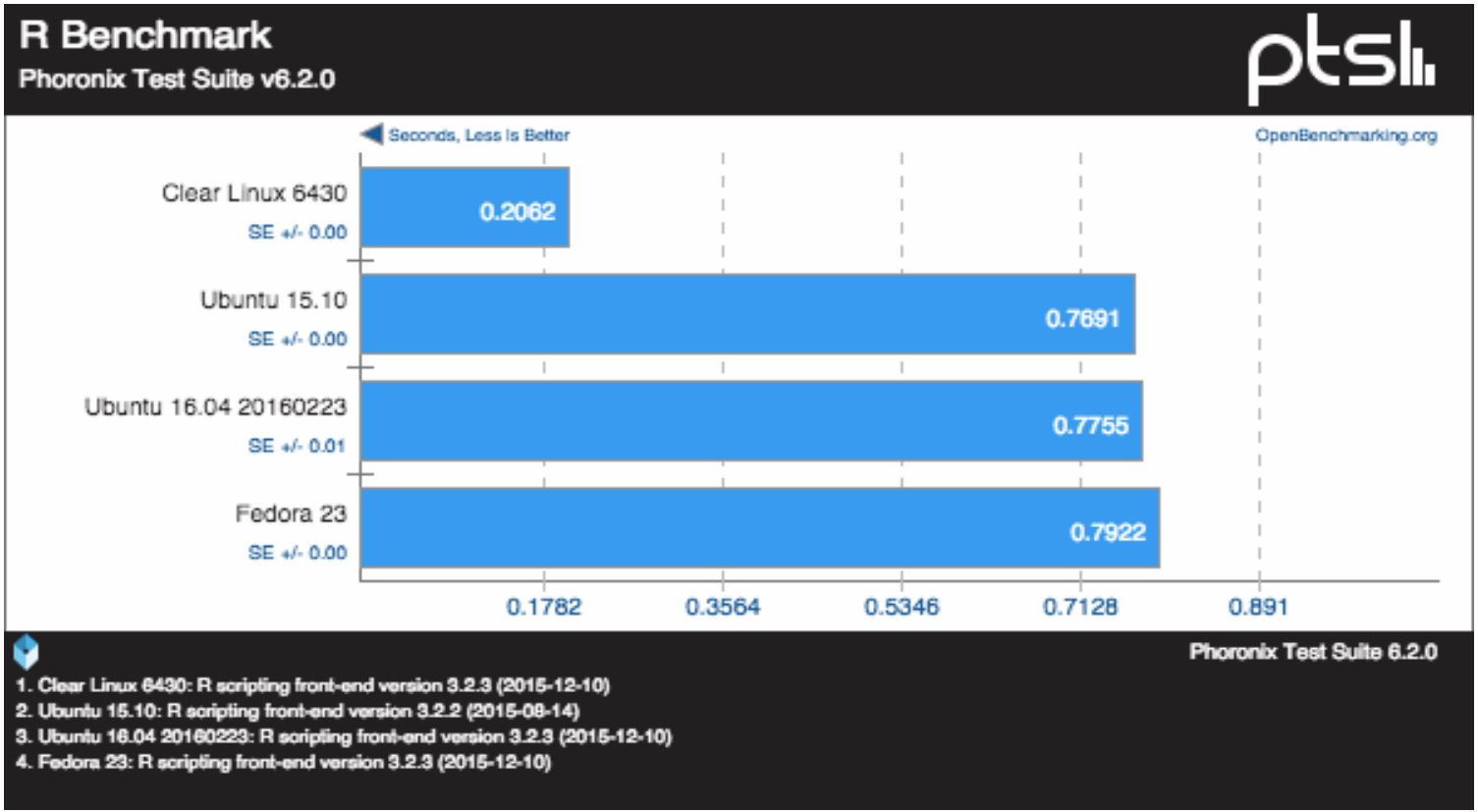
```
%files
%defattr(-,root,root,-)
/usr/include/cblas.h
/usr/include/f77blas.h
/usr/include/lapacke.h
/usr/include/lapacke_config.h
/usr/include/lapacke_mangling.h
/usr/include/lapacke_utils.h
/usr/include/openblas_config.h
/usr/lib64/avx2/libopenblas.a
/usr/lib64/avx2/libopenblas.so
/usr/lib64/avx2/libopenblas.so.0
/usr/lib64/avx2/libopenblas_haswellp-r0.2.16.a
/usr/lib64/avx2/libopenblas_haswellp-r0.2.16.so
/usr/lib64/libopenblas.a
/usr/lib64/libopenblas.so
/usr/lib64/libopenblas.so.0
/usr/lib64/libopenblas_sandybridgep-r0.2.16.a
/usr/lib64/libopenblas_sandybridgep-r0.2.16.so
/usr/lib64/avx2/cmake/openblas/OpenBLASConfig.cmake
/usr/lib64/cmake/openblas/OpenBLASConfig.cmake
/usr/lib64/avx2/cmake/openblas/OpenBLASConfigVersion.cmake
/usr/lib64/cmake/openblas/OpenBLASConfigVersion.cmake
```

Real case : example 1




```
%configure --disable-static
        --without-x
        --with-system-zlib
        --with-system-bzlib
        --with-system-pcre
        --with-system-xz
        --enable-BLAS-shlib
        --enable-R-shlib
        --with-blas="-lopenblas"
        --with-cairo --enable-lto
```

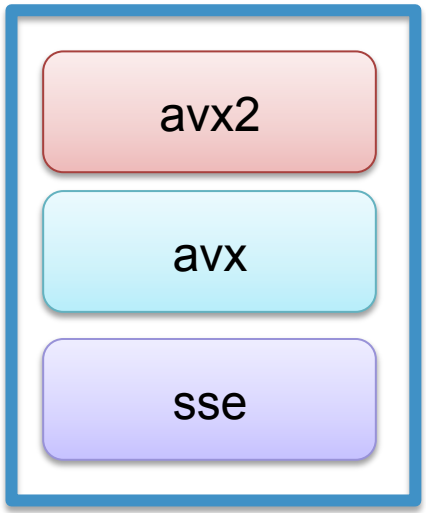
Real case : example 1



Problem ?



Why don’t we put everything in the same box



FMV How does it works ?



http://lwn.net/Articles/691932/

In the old times
*and **only** in C++*
(before GCC 6)

```
__attribute__ ((target ("sse4.2")))
int foo(){
    // foo version for SSE4.2
    return 1;
}
__attribute__ ((target ("arch=atom")))
int foo(){
    // foo version for the Intel Atom processor
    return 2;
}

int main() {
    int (*p)() = &foo;
    assert((*p)() == foo());
    return 0;
}
```

The target() directives will compile the functions for instruction-set extensions (e.g. sse4.2) or for specific architectures (e.g. arch=atom).X

- Here, for each function, the developer needed to create specific functions and code for each target.

- That would have required extra overhead in the code; increasing the number of LOC in a program for FMV makes it more **clunky** to manage and maintain.



- Fortunately, GCC 6 solves this problem: it supports FMV in both **C and C++** code with a **single attribute** to define the **minimum** set of **architectures** to support.

- This makes it **easier** to develop Linux applications that can take **advantage** of **enhanced** instructions, **without** the **overhead** of **replicating** functions for each target.

Thanks to !!!
Evgeny Stupachenko



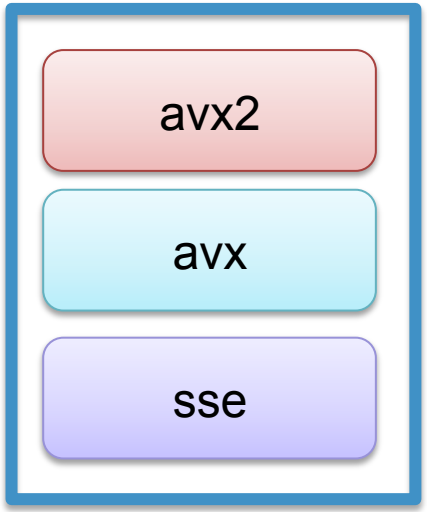
Image taken of : http://www.picgifs.com/graphics/t/tux/graphics-tux-510908.gif

Function Multiversioning (FMV)

```
#define MAX 1000000
   int a[256], b[256], c[256];

   __attribute__((target_clones("avx2","arch=atom","default")))
   void foo(){
       int i,x;

    for (x=0; x<MAX; x++){
        for (i=0; i<256; i++){
         a[i] = b[i] + c[i];
        }
    }
   }
   int main() {
       foo();
       return 0;
   }
```



FMV

We will build that with:

```
$ gcc -O3 sanity.c -o sanity
```

And you can watch the magic with objdump like this:

```
gcc -O3 sanity.c -g -c
objdump -S sanity.o
```

FMV

No AVX code (Atom):

```
add     %eax,%edx
```

AVX:

```
vpaddd 0x0(%rax),%xmm0,%xmm0
```

AVX2:

```
vpaddd (%r9,%rax,1),%ymm0,%ymm0
```

CPUID selection

- In GCC 4.8, FMV had a dispatch priority rather than a CPUID selection.
- Function versions with more advanced features got higher priority.
- For example, a version targeted for AVX2 would have a higher dispatch priority than a version targeted for SSE2.

    - In GCC 6, the resolver checks the CPUID and then calls the corresponding function. It does this once per binary execution.
    - So when there are multiple calls to the FMV function, only the first call will execute the CPUID comparison; the subsequent calls will find the required version by a pointer.
    - This technique is already used for almost all glibc functions. For example, glibc has memcpy() optimized for each architecture, so when it is called, glibc will call the proper optimized memcpy().

Results

	Execution Time (ms)					
GCC flags	**Haswell**	**Skylake**	**Broadwell**	**Xeon**	**Atom**	**Ivy Bridge**
None	603	645	580	1413	2369	517
`-O3`	38	44	37	107	96	60
`-O3 -mavx`	26	32	26	73	`SIGILL`	45
`-O3 -mavx2`	26	32	26	73	`SIGILL`	`SIGILL`
`-O3` (with FMV)	26	32	26	73	96	45

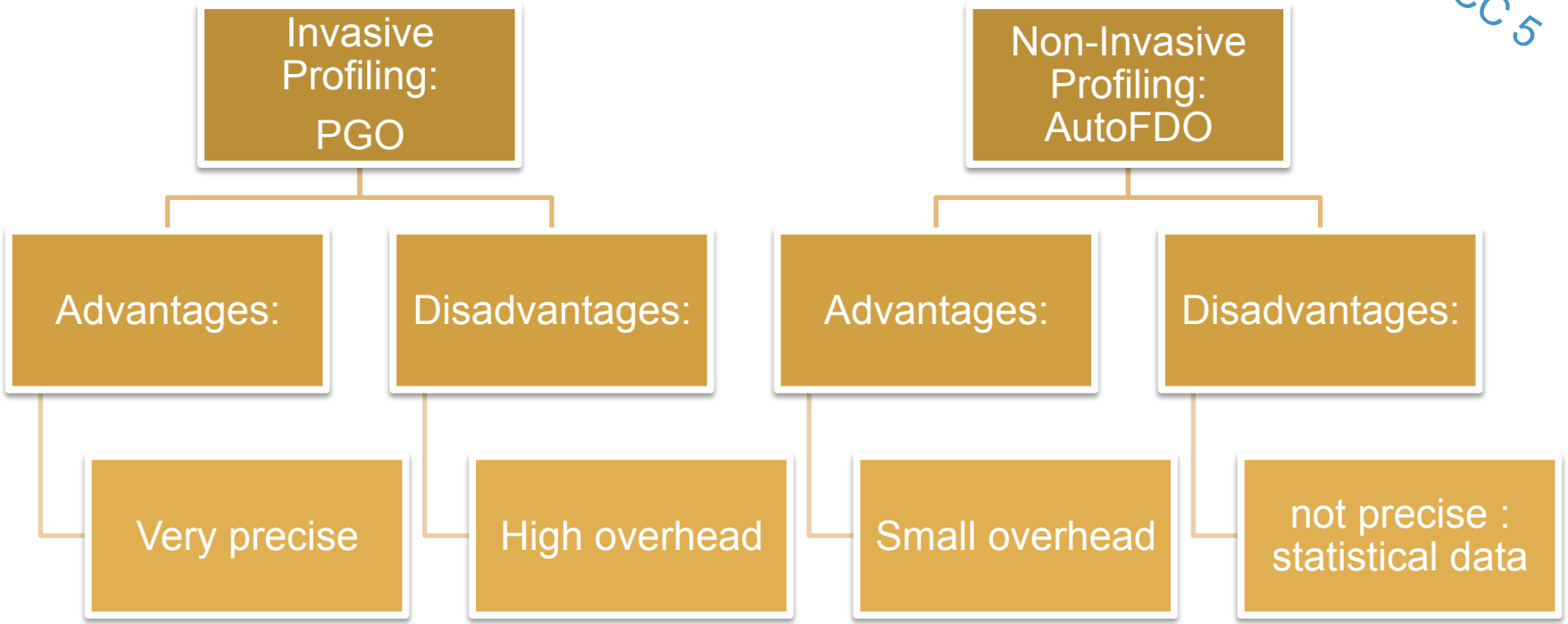
Profiling…

Helping the compiler find the most efficient path...

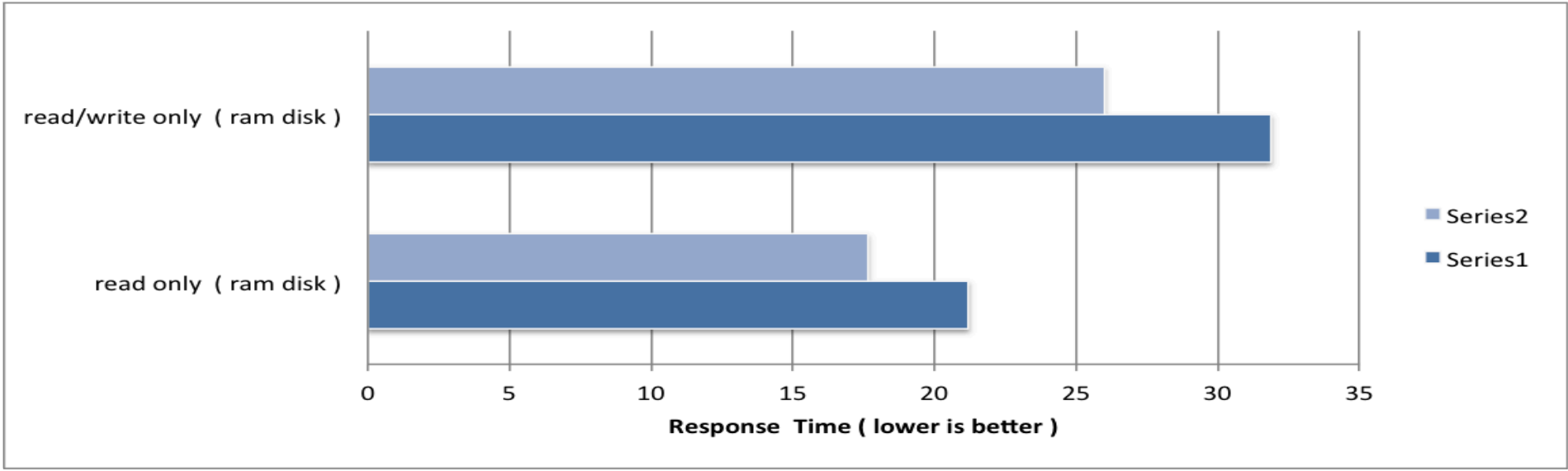


Profiling.. Learning on the road

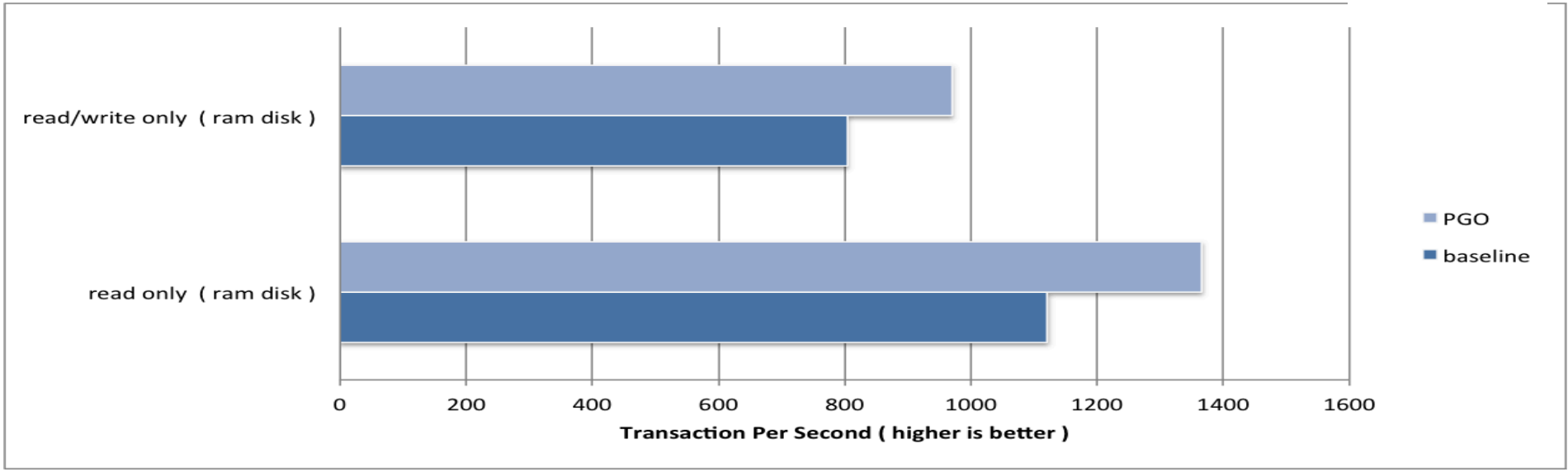
Since GCC 5



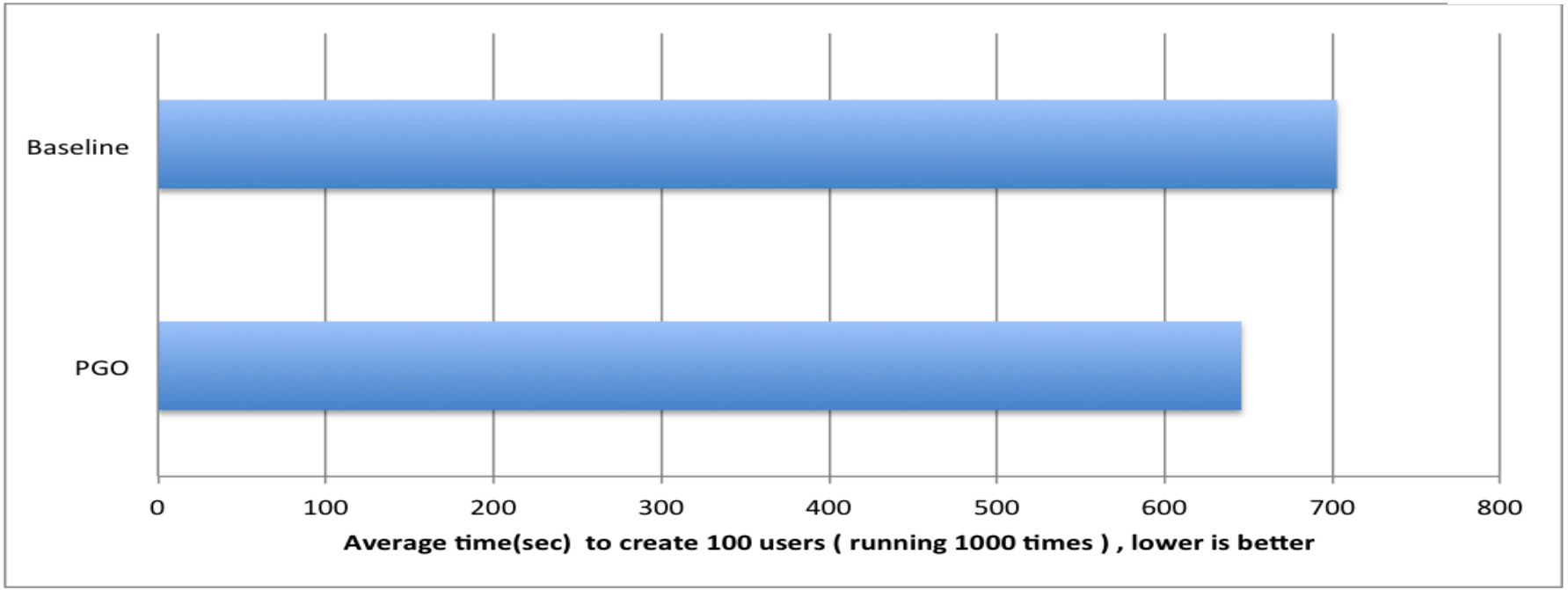
MariaDB + PGO


read/write only (ram disk)
read only (ram disk)
0
5
10
15
20
25
30
35
Response Time (lower is better)
Series2
Series1

MariaDB + PGO


read/write only (ram disk)
read only (ram disk)
0
200
400
600
800
1000
1200
1400
1600
Transaction Per Second (higher is better)
PGO
baseline

MariaDB + PGO + Open Stack



Phase 1: Generate the profile file

AutoFDO needs a perf.data file that captures the BR_INST_RETIRED:TAKEN event in the processor. This event will vary for every architecture, so we are going to use ocperf, which is part of the pmu-tools project, which wraps all the information required for perf to generate the perf.data correctly for any Intel architecture. The user is free to use this tool or just the perf tool.

```
# ocperf.py record -b -e br_inst_retired.near_taken:pp -- ./sort
Bubble sorting array of 30000 elements
3731 ms
[ perf record: Woken up 7 times to write data ]
[ perf record: Captured and wrote 1.580 MB perf.data (3902 samples) ]
```

After this a standalone tool is used to convert the perf.data file into gcov format. This tool is create_gcov from autofdo set of tools:

```
# create_gcov --binary=./sort --profile=perf.data --gcov=sort.gcov -gcov_version=1
```

Phase 2: Use profile to optimize binary

The following info is read from the profile gcov file (in our case sort.gcov) :

- Function names and file names.
- Source level profile, which is a mapping from inline stack to its sample counts.
- Module profile: Module to aux-modules mapping

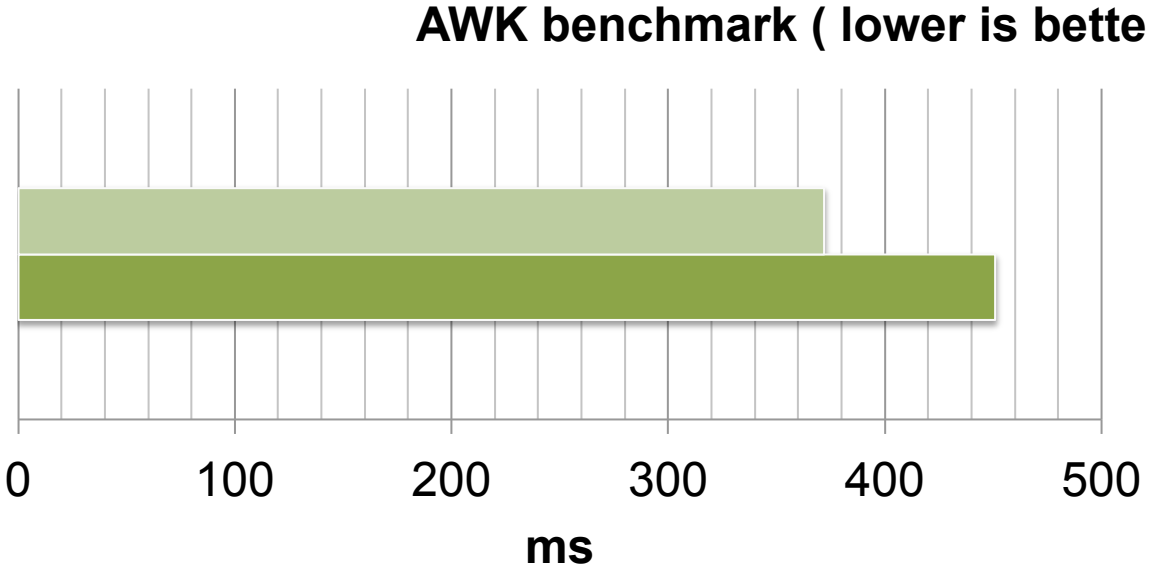
In order to read the profile file we need to rebuild the source:

```
# gcc -O3 -fauto-profile=sort.gcov sort.c -o sort_autofdo
```

After this we have the binary sort_autofdo, the which we can run to test:

https://gcc.gnu.org/wiki/AutoFDO/Tutorial

AWK and AutoFDO ?

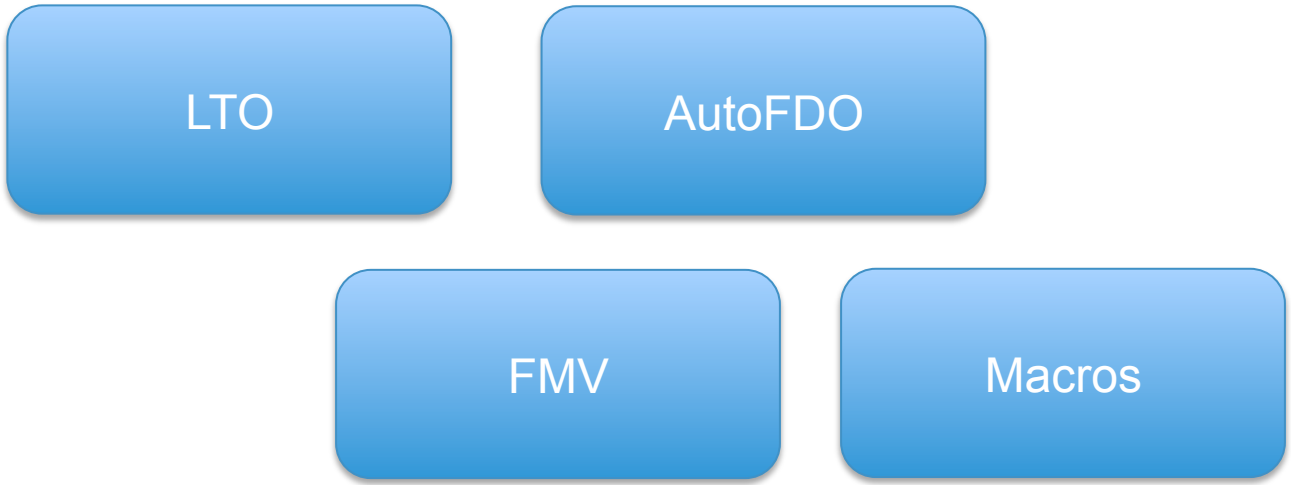


https://gcc.gnu.org/wiki/AutoFDO/Tutorial

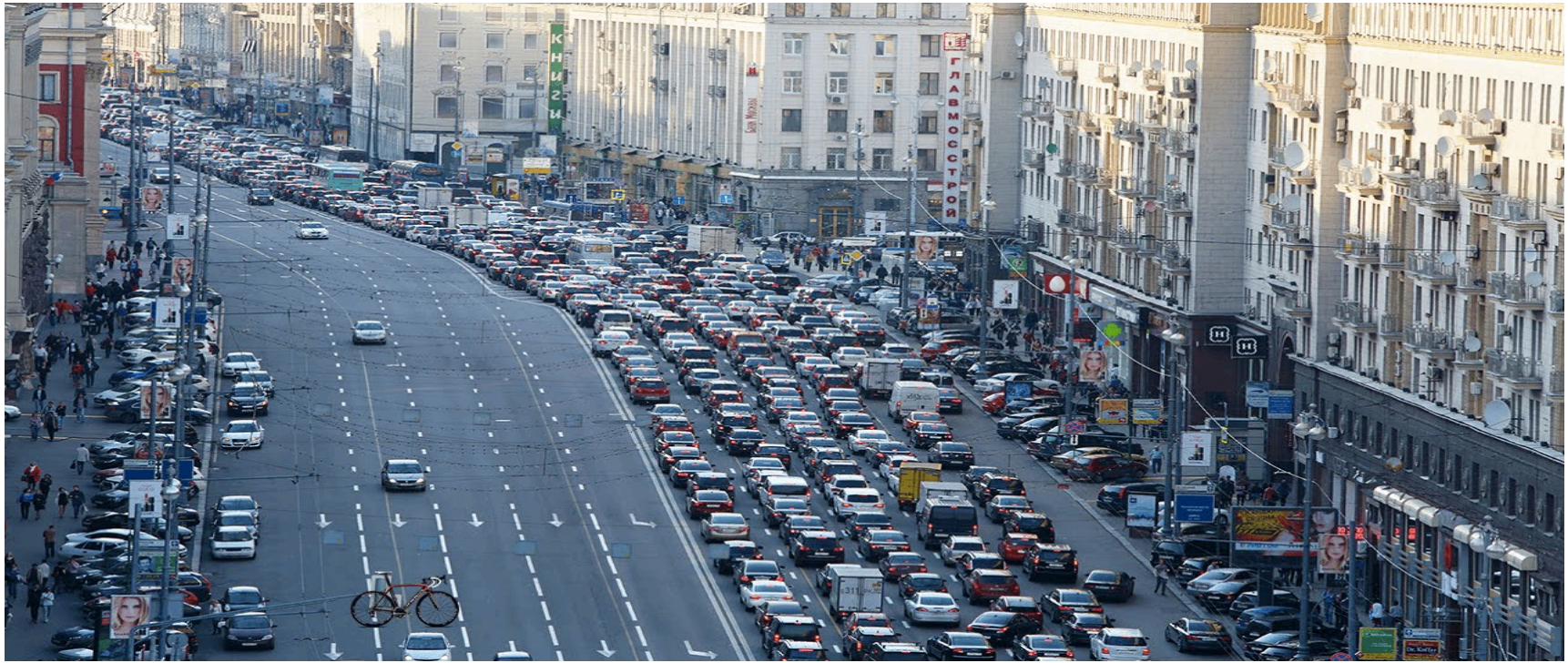
Thanks to !!!
Andi Kleen



Bugs .. Not everything was pretty



Unused resources….



https://i.ytimg.com/vi/X0ymEx64RLw/maxresdefault.jpg


Thank
you